

A Multi-Armed Bandit Approach for Test Case Prioritization in Continuous Integration Environments

Jackson A. Prado Lima  and Silvia Regina Vergilio 

Abstract—Continuous Integration (CI) environments have been increasingly adopted in the industry to allow frequent integration of software changes, making software evolution faster and cost-effective. In such environments, Test Case Prioritization (TCP) techniques play an important role to reduce regression testing costs, establishing a test case execution order that usually maximizes early fault detection. Existing works on TCP in CI environments (TCPCI) present some limitations. Few pieces of work consider CI particularities, such as the test case volatility, that is, they do not consider the dynamic environment of the software life-cycle in which new test cases can be added or removed (discontinued), characteristic related to the Exploration versus Exploitation (EvE) dilemma. To solve such a dilemma an approach needs to balance: i) the diversity of test suite; and ii) the quantity of new test cases and test cases that are error-prone or that comprise high fault-detection capabilities. To deal with this, most approaches use, besides the failure-history, other measures that rely on code instrumentation or require additional information, such as testing coverage. However, to maintain the information updated can be difficult and time-consuming, not scalable due to the test budget of CI environments. In this context, and to properly deal with the TCPCI problem, this work presents an approach based on Multi-Armed Bandit (MAB) called COLEMAN (*Combinatorial VOLatile Multi-Armed BANDit*). The TCPCI problem falls into the category of volatile and combinatorial MAB, because multiple arms (test cases) need to be selected, and they are added or removed over the cycles. We conducted an evaluation considering three time budgets and eleven systems. The results show the applicability of our approach and that COLEMAN outperforms the most similar approach from literature in terms of early fault detection and performance.

Index Terms—Test case prioritization, continuous integration, multi-armed bandit

1 INTRODUCTION

CONTINUOUS Integration (CI) plays an important role in Agile development, allowing reduced integration effort, lower number of uncorrected errors for long periods, and delivering a product version at any moment. CI environments are fundamental to CI in practice to support the frequent integration of changes and make the software evolution more rapid and cost-effective. Studies in the literature show that the adoption of such environments in the industry is growing, as well as the number of daily commits and the amount of automated tests as a consequence [1].

Companies like Google [2], Facebook, and Microsoft have adopted CI, as well as open-source projects [3] using available CI frameworks (i.e., Travis CI and Jenkins). A study shows that every day at Google, an amount of 800K builds, and 150 Million test runs are performed on more than 13K code projects [2]. This amount of builds and tests can require non-trivial amounts of time and resources [3]. Within an integration cycle (many times called build), the regression testing activity takes a significant amount of

time. A test set, many times, includes thousands of test cases that take several hours or days to execute [4]. Even though massive parallelism was reported, Google developers must wait 45 minutes to 9 hours to receive testing results [2].

In this scenario, to re-execute all test cases is unfeasible. Then it is fundamental to perform Regression Testing (RT) activities in a very cost-effective way. This ensures that recent changes have not negatively impacted functionality previously tested, and considering CI goals, provides rapid test feedback on software failures. Besides, in a company, multiple projects may share the same CI workflow and regression testing usually runs a time restricted to a specific duration, the test budget. This makes difficult the use of traditional RT techniques for test case minimization and selection. They usually rely on code analysis and instrumentation, what can be time-consuming and produce results that quickly become inaccurate due to the frequent changes [5]. Test Case Prioritization (TCP) techniques are most suitable in the presence of time constraints. TCP techniques allow the most crucial test cases are executed first, by reordering a test suite according to specific goals, such as early fault detection. TCP techniques have advantages with respect to other techniques because they consider all test suite, consequently, decreasing the risk of reducing code coverage by discarding some test case [6].

Ideally, the test set should be executed to maximize early fault detection. However, fault-detection information is unknown until the testing is finished [7]. TCP techniques based on failure history can be used to overcome such a difficulty

- The authors are with the Department of Informatics, Federal University of Paraná (UFPR), Curitiba, PR 80060-000, Brazil.
E-mail: jacksonpradolima@gmail.com, silvia@inf.ufpr.br.

Manuscript received 2 Oct. 2019; revised 19 Apr. 2020; accepted 29 Apr. 2020.
Date of publication 4 May 2020; date of current version 14 Feb. 2022.
(Corresponding author: Jackson A. Prado Lima.)
Recommended for acceptance by Y. Brun.
Digital Object Identifier no. 10.1109/TSE.2020.2992428

[4]. But a problem remains, related to the speed up of existing TCP approaches. Due to the high frequency of changes in CI environments and the high RT cost, approaches that require exhaustive analysis are costly and inefficient [5] because the time available to run the prioritized test suite can be reduced if prioritization takes too long [8].

To deal with the TCP problem in CI environments (TCPCI), some approaches have recently appeared in the literature addressing such difficulties: early fault detection and time constraints to run the test suite [4], [9], [10], [11], [12], [13], [14]. But few pieces of work consider CI particularities [15]. For instance, they do not consider the dynamic environment of the software life-cycle, in which changes in the code are frequent and test cases can be added or removed (discontinued) over the CI cycles, particularity called test case volatility. Most approaches do not properly deal with an important dilemma called Exploration versus Exploitation (EvE). To solve such a dilemma an approach needs to balance: i) the diversity of test suite, and ii) the quantity of new test cases and test cases that are error-prone or that comprise high fault-detection capabilities. This problem is related with the test budget, because if only error-prone test cases are considered without diversity, some test cases can never be executed. To deal with this, existing approaches use, besides the failure-history, other measures that rely on code instrumentation or require additional information, such as to calculate code or feature coverage. This can be time-consuming, and to maintain the information updated can be difficult.

The approach called RETECS (Reinforced Test Case Selection) [14] deals with the EvE dilemma and considers the test budget by using historical test data and Reinforcement Learning (RL) [16]. RETECS reached the best performance using an Artificial Neural Network (ANN), which is capable of handling a large amount of decisions/states. On the other hand, determining why an ANN makes a particular decision is a hard task (black box) [17].

Considering the aforementioned limitations of existing work, in this paper, we introduce an approach for the TCPCI problem based on Multi-Armed Bandit (MAB) [18], [19]. The MAB problems are a class of sequential decision problems that are intensively studied for solving the EvE dilemma [20]. MAB has many similarities to RL and is considered to be a “lite” form (one-state) of RL. But MAB presents some advantages. MAB does not require context information and its actions only affect the reward, that is, the actions do not change the state of the environment. In contrast, RL actions change the state. Due to this, RL needs to handle the state space, as well as to rely on function approximation to evaluate the value of being in a particular state and taking a specific action.

The TCPCI problem falls into the category of volatile and combinatorial MAB, because multiple arms (test cases) need to be selected, and they are added or removed over the cycles. Then, we named our approach COLEMAN (*Combinatorial VOLatile Multi-Armed BANDit*). We implemented our approach with five MAB policies and evaluated it in eleven large-scale real-world software systems, considering three time budgets. After that, we compared the best policy found for COLEMAN against RETECS [14]. Quality indicators, such as Normalized Average Percentage of Faults Detected

(NAPFD), Average Percentage of Faults Detected with cost consideration (APFDc), as well as statistical tests and effect size, are used. Evaluation results show the applicability of our approach, which takes in all cases less than one second to execute, even for the systems with the greatest number of builds and test cases. COLEMAN outperforms RETECS with statistical difference in the majority of the cases, considering all budgets and indicators used. In summary, this work has the following main contributions:

- 1) The proposal of COLEMAN, a MAB-based approach for the TCPCI problem. Such an approach has some advantages in comparison with related work:
 - It learns how to incorporate the feedback from the application of the test cases thus incorporating diversity in the test suite prioritization;
 - It uses a policy to deal with the EvE dilemma, thus mitigating the problem of beginning without knowledge (learning) and adapting to changes in the execution environment, for instance, the fact that some test cases are added (new test cases) and removed (obsolete test cases) from one cycle to another (volatility of test cases);
 - It is model-free. The technique is independent of the development environment and programming language, and does not require any analysis in the code level;
 - It is more lightweight, that is, needs only the historical failure data to execute, and has higher performance.
- 2) As far as we know, COLEMAN is the first MAB-based approach that considers standard MAB with the characteristics of combinatorial and volatile MAB, even considering other fields beyond software testing.
- 3) A public repository with the data used in this work, which allows replication and can be used in future research available in the Open Science Framework [21].

Paper Structure. Section 2 provides background information about MAB and CI environments. Section 3 reviews related work on TCPCI. Our proposed approach, COLEMAN, is detailed in Section 4. Section 5 describes how the experiments were conducted: research questions, systems under test, quality indicators and used parameters. Results are presented and discussed in Section 6. Section 7 concludes the paper and presents the directions for future work.

2 BACKGROUND

In this section, we review background on Combinatorial and Volatile Multi-Armed Bandit and CI environments.

2.1 Multi-Armed Bandit

MAB (*Multi-Armed Bandit*) problems [18] are sequential decision problems related to the EvE (Exploitation versus Exploration) dilemma [20]. This means that, for such problems, solutions with the best performance (exploitation) are desired, but it is also important to ensure diversity, that is, dissimilar solutions (exploration).

The MAB problem is related to the scenario in which a player plays on a set K of slot machines (or arms/actions)

that even identical produce different gains. After a player pulls one of the arms $i \forall i \in K$ in a turn t , a reward $(\hat{q}_{i,t})$ is received drawn from some unknown distribution, thus aiming to maximize the sum of the rewards.

A policy is a strategy that chooses, at each time t , the next arm to pull based on previously observed rewards and decisions. The MAB problem is to determine the policy that maximizes the expected cumulative reward over the EvE dilemma. A review of the main MAB policies proposed in the literature is presented in [20]. Among them, we can mention the ϵ -greedy policy, a policy widely used due to its simplicity. At each time, such a policy evaluates the arms and defines an empirical quality estimate $\hat{q}_{i,t}$ based on previous executions. The $\hat{q}_{i,t}$ value of an arm i in the time t is based on the sum of its previous rewards divided by the number of times that i has been pulled. After, the policy selects, with a probability $1 - \epsilon$, the arm with the highest $\hat{q}_{i,t}$ value (exploitation), or randomly selects an arm (exploration) with a probability ϵ . The parameter ϵ is the key to balance the EvE dilemma in the ϵ -greedy policy.

The MAB-policy called Upper Confidence Bound (UCB) provides a smarter way to deal with the EvE dilemma and ensures asymptotic optimality in terms of the total cumulative reward [19]. Most of the recent policies are UCB-based. In such policies the i th arm has an empirical quality estimate $\hat{q}_{i,t}$ (the average of the rewards obtained up to the given time instant) and a confidence interval that depends on the number of times, n_i , the arm has been applied before. At each time point t , the selection of the best arm is performed based on the arm with the best upper bound of the confidence interval. In addition to this, Fialho [22] introduced a scaling factor C (Equation 1), encompassing the situation when the rewards are usually among some real-value interval and the EvE balance may “break”.

$$\text{Select } a_t = \underset{i \in K}{\operatorname{argmax}} \left(\hat{q}_{i,t} + C \times \sqrt{\frac{2 \times \ln \sum_{j=1}^K n_{j,t}}{n_{i,t}}} \right), \quad (1)$$

where the exploitative first term favors the arms with best empirical rewards, while the exploratory second term favors the infrequently tried arms. If exploration is preferable, C must be increased. Otherwise, if exploitation is the focus, C must be decreased. In this work, the acronym UCB is used to denominate the adaptation proposed by Fialho.

Another UCB-based MAB policy is the Fitness-Rate-Rank (FRRMAB), a state policy that has presented good results in the Adaptive Operator Selection context [23]. This policy consists of two procedures: credit assignment and operator (arm/action) selection. The first procedure, *Credit Assignment*, refers to a reward procedure that takes into account the impact observed in the most recent applications. In the credit assignment, FRRMAB policy changed the UCB quality estimator (Equation (1)) by a rank-based method that uses the Fitness Improvement Rate (FIR) method. The FIR value is stored in a given Sliding Window (SW) organized as a first-in, first-out queue that is used to evaluate the W recent applications. The FRRMAB final value is corrected by a decaying factor (DF). The second procedure randomly

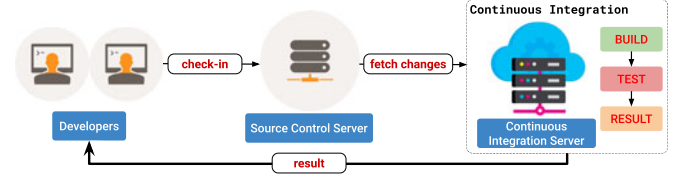


Fig. 1. Overview of a Continuous Integration environment.

selects an arm until all arms are selected, uses the FRRMAB policy to evaluate each arm, and selects the best one.

In many real-world scenarios, a policy needs to select multiple arms in each time. This is the case in our scenario, TCP CI problem, where we need to prioritize a test set, and we can consider a test case as an arm. This kind of MAB problem is categorized as *combinatorial bandit*, in which a set of arms are chosen in each time t rather than one individual arm, that is, we are required to pull a fixed number m of arms from a set of arms K , such that $1 \leq m \leq |K|$ [24].

In addition to this and considering the inherently dynamic nature of our problem, the arms available in each time may change dynamically over time. In this sense, this work is based on a MAB variant known as *Volatile-multi-Arm bandit* (VMAB) [25]. VMAB considers that the arms can “appear” or “disappear” unexpectedly in each time. In VMAB each arm a_i is associated with a *lifespan* given by an interval of time (t_{start_i}, t_{end_i}) , during which this arm is available. The arm’s *lifespans* are unknown in advance.

In this paper, we propose a novel approach that combines combinatorial MAB and VMAB. It allows the use of multiple plays in an adaptive (volatile) scenario.

2.2 Continuous Integration Environments

We observe an increasing use of Continuous Integration (CI) environments in the industry [26]. CI environments automate the process of building and testing software, allowing engineers to merge code that is under development or maintenance with the mainline code base at frequent time intervals [26]. In CI development, teams work continuously integrating code and make smaller code commits every day, usually monitored by a CI server. When a change occurs, the CI server clones this code, builds it and runs the testing processes. When the entire process is finished, a report (feedback) is generated by the CI server, and the developers are informed. Fig. 1 illustrates the process.

CI automated support is very important to CI in practice. Zhao *et al.* [1] present some results about the impact of adopting the framework Travis CI on development practices in a collection of GitHub projects. They observed an increase in the number of daily commits (frequency of 78 commits every day) and, after some initial adjustments, an increase in the amount of automated tests.

In this scenario, RT activities are fundamental and TCP techniques have been adopted to deal with the test budget, limited resources and constraints of CI environments. In the next section, we review related work on TCP CI.

3 RELATED WORK

There are many works on TCP in the literature, which have been subject of surveys and mappings [7], [27], [28].

However, only few pieces of work address CI environments. Some approaches have the goal of reducing the amount of resources utilized in a CI environment. Liang *et al.* [29] propose an approach to prioritize commits based on the test suite failure and execution history. Other approach that considers multiple test requests was proposed by Zhu *et al.* [30]. Such an approach uses co-failure distributions of tests, that is, tests that co-fail in previous executions in different sets. Elbaum *et al.* [5] introduce new algorithms for selection and prioritization of test suites, to be used respectively in a pre-submit and pos-submit testing phases. In the first phase, which occurs prior to commit, developers specify modules to be tested and the selection algorithm selects test sets based on failure and execution windows. This allows, respectively, selection based on the fault detection history and of test cases not recently executed. In the post-testing, after commit, the TCP algorithm prioritizes test suites considering both windows and a time window. In this way, the algorithm deals with test suite concurrency execution in the pos-submit testing.

Differently from the mentioned works, our work assumes a sequential execution order of test cases and prioritizes test cases in the suite to be executed after the commit, usually considering a test budget. Approaches with such a goal are the most related to ours, and are described next.

Marijan *et al.* [9] introduce ROCKET, an approach that, given a test budget, sets a weight for each test case based on the distance of the failure status from its current execution and its execution time. We observe a limitation regarding the execution time. Test cases with an execution time greater than a limit are penalized, and it is possible they are never executed. To set the weight, neither the prioritization feedback is considered nor the total history of failures. An extension is proposed by the authors in [10] to consider other perspectives regarding fault detection, business, performance and technical aspects. Such an extension needs additional information related to coverage and features.

An approach and a tool called TITAN is proposed in [11] for Highly Configurable Software (HCS). It implements test prioritization and minimization techniques, and provides test traceability and visualization. Again, additional information, such as feature coverage, is necessary. As first a minimization step is conducted the prioritized set may not contain all available test cases.

Cho *et al.* [12] proposed an approach, named AFSAC, composed by two stages. First, weights for the test cases are determined by using statistical analysis over the failure history. Then, the test cases are reordered using the correlation data of test cases acquired by previous test results.

Haghighatkhan *et al.* [4] present empirical results that show the use of historical failure knowledge is a strong predictor for TCPCI problem. It is effective to catch regression faults earlier without requiring a large amount of historical data. In addition to this, the effectiveness can be improved by using such a knowledge with a diversity measure, calculated by comparing the text of test cases.

The above-mentioned approaches do not consider the volatility of the test cases, that is, they do not consider a test case may added and/or removed over the cycles. Most of them do not consider the feedback from the prioritization conducted previously. Some works have addressed this

limitation with the use of machine learning. The approach of Busjaeger and Xie [13] uses (SVM^{map}) to create a model based on five attributes: test coverage of modified code, textual similarity between tests and changes, recent test-failure or fault history, and test age. Such a model is used to predict the fault-proneness of the test cases and prioritize them. However, to obtain the used attributes, additional information and code instrumentation are necessary. Other approach is RETECS, introduced by Spieker *et al.* [14] to prioritize and select test cases based on Reinforcement Learning (RL). RETECS considers as input the test case duration, historical failure data, and previous last execution. The authors compared different RL variants and the Artificial Neural Network (ANN) variant presented the best results.

We can summarize some drawbacks identified in related work. Some of them have different goals from ours, for instance to reduce server resources considering concurrent test set executions. The most related approaches do not properly deal with the EvE problem. This problem regards to the fact that as only a sub-set of the prioritized test cases can be executed regarding its order, some test cases can never be executed given the test budget. To deal with this, most approaches use, besides the failure-history, other measures that rely on code instrumentation or require additional information, such as to calculate code or feature coverage. This can be time-consuming and to maintain the information updated can be difficult.

Existing works do not take into consideration the volatility of test cases and feedback from last prioritizations. Differently, our approach considers the test cases volatility and learns with the past prioritizations (*online learning*). It properly deals with the EvE dilemma without requiring source code analysis or any initial concept (model) about the system. In this sense, RETECS [14] is the most similar to ours. But RETECS has the best performance using an ANN [14], which usually requires a large amount of data [31]. Moreover, determining why an ANN makes a particular decision is a hard task. This makes difficult to trust in their reliability for real-world problems [17]. Our approach uses MAB, which allows test cases rewards in a sliding window, and works with less input information. Context information is not necessary. In this way, we aim to properly deal with the EvE dilemma with an approach more lightweight and with higher performance.

4 PROPOSED APPROACH

In this section we describe our test case prioritization approach, named COLEMAN (*Combinatorial VOLatile Multi-Armed BANdit*). Given the dynamic nature of our problem, our MAB approach combines two MAB variants: i) combinatorial MAB, because we have a set of arms (test cases), and ii) volatile, because at a given time t such set varies, test cases can be added or removed over the software life-cycle. In addition to this, the approach works with a budget (constraint) to execute the test cases prioritized. To ensure diversity of test cases to be executed, it uses MAB policies, allowing better exploitation and exploration (EvE dilemma).

Fig. 2 illustrates how our approach works in a CI environment. After a successful build, in the test phase, the approach receives as input a set of test cases T_i (arms)

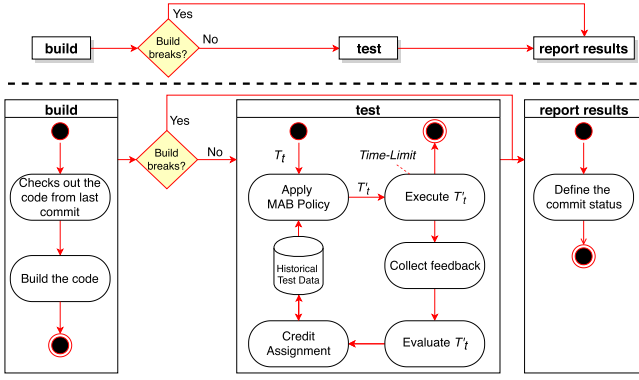


Fig. 2. Overview of the proposed approach and how it is integrated in the test phase of a CI environment.

available for the current commit (cycle/time) t and, based on the choice order given by a MAB policy, generates the prioritized test case set T'_t . In each time t , only one test suite (test case set) is prioritized. Then the system is tested using T'_t and feedback from this test set is collected, containing information, such as the test cases executed in a time limit, the number of failures, the test case failure rank, and so on. This feedback is used by the reward function in the credit assignment procedure to set individual rewards for each test case. In the T'_t evaluation, a fitness value of T'_t is obtained by a quality indicator.¹ This value can be used by testers along with the commits to evaluate the prioritization quality.

Then, the credit assignment procedure calculates the rewards for each arm, test case $t'_c \in T'_t$, which are stored in a historical database to be used in the next commits by the policies. In the end, results are reported back to the CI server. The CI server sets the result from the cycle and notifies the parties interested in the cycle.

Next, we detail the main elements of COLEMAN: MAB policies (Section 4.1) and credit assignment (Section 4.2).

4.1 MAB Policies

Considering the traditional behavior of a MAB policy, the best arm (test case) t_c from a set T_t at each time t is chosen and applied. However, we are working with TCP problem where a prioritized test set is used. To this end, we can adapt a policy to choose the best arm (test case) t_c from T_t to compose T'_t , then remove t_c from T_t , and continue this process until no more test cases are available in T_t . An arm t_c is chosen only once and the order of choice defines the prioritization execution. But this process of choice is costly, once that a new evaluation for each test case in T_t is necessary to choose the next best test case to compose T'_t .

To reduce the selection cost, we adapted the policies to evaluate all the test cases (arms), and order them, putting the best test case in the top, followed by the second best one, and so on. When more than one test case has the same performance, the order among them is defined randomly. This simple adaption allows a fast prioritization whilst considers the characteristics of the policy chosen. We also adapt the chosen policy to consider only the test cases available in time t and ignore the other ones from the previous time.

This modification allows us to consider the dynamic environment (volatility) of the test cases.

As mentioned in Section 2.1, there are many MAB policies. We chose the policies that better work with the EvE dilemma: ϵ -greedy, UCB, and FRRMAB. Besides these policies, we also assessed the following policies as baselines: i) Random: is a strategy (not a real strategy per se) where the player will only do exploration; and ii) Greedy: only takes the best apparent arm, and it is a special case of ϵ -greedy where $\epsilon = 0$, i.e., it always does exploitation.

It is important to highlight that FRRMAB policy works with a sliding window. The reward value (FIR for FRRMAB) is obtained through a reward function (Section 4.2.1), then the last W rewards are used by the policy. In this way, for each test case, FRRMAB policy considers the history of rewards whilst the other ones use cumulative rewards.

4.2 Credit Assignment

This procedure reflects the goal of the prioritization and teaches the MAB-based policy about the test cases considering historical test data. In this procedure, for $t'_c \forall t'_c \in T'_t$, two values are assigned: the number of times $n_{t'_c}$ has been applied before and the reward value ($\hat{q}_{t'_c,t}$). These values are used by the policies to generate the prioritized test set. At the beginning (where $t = 1$) and for each new test case, the values for $n_{t'_c}$ and $\hat{q}_{t'_c,t}$ are assigned with zero. After that, the values are assigned as follows.

The number of times $n_{t'_c}$ that t'_c has been applied before is considered to explore new test cases (few used). Traditionally, a MAB policy selects an arm and increments the number of times that this arm was chosen. In our case, we use a combinatorial MAB, and this kind of MAB selects a set of arms (test cases). To counterbalance the order of choice, a weight is given for each $t'_c \in T'_t$ according to its order in T'_t . The weights are evenly spaced values within an interval (0.0, 1.0) with a step size of $\frac{1}{|T'_t|}$ in descending order. In this way, the highest weight is given to the first test case in T'_t and the lowest to the last one. Then, $n_{t'_c}$ is incremented with the weight defined to t'_c .

The reward value is obtained by a reward function (Section 4.2.1). This value is used to exploit the best test cases. As described in Section 4.1, the reward value is stored in a sliding window when FRRMAB policy is used whilst ϵ -greedy, greedy, and UCB policies use a cumulative reward strategy. If a new test case appears, a zero is set for the reward value, once that we do not have a *test case history*. On the other hand, if a test case is removed in the current cycle (commit), we remove its history.

4.2.1 Reward Functions

In this work, we adopt and adapted two reward functions from related work [14]. The first reward function *RNFail* (Reward Based on Failures) is based on the number of failures associated with a test case $t'_c \in T'_t$ and uses the function *fails* defined in Table 1.

$$RNFail(t'_c) = \begin{cases} 1 & \text{if } fails(t'_c) \\ 0 & \text{otherwise} \end{cases} \quad (2)$$

The second reward function *TimeRank* (Time-Ranked Reward) is based on the rank of $t'_c \in T'_t$ (Equation (3)). The

1. In this work we use NAPFD (see Section 5.1).

TABLE 1
Functions Used by the Reward Functions

Definition	Description
$failures(t'_c)$	In our context, a test case t'_c can be composed by many parts (or test methods), each one of this part can be associated with a failure. In this way, a failing test case t'_c can be associated with one or more failures. Function $failures(t'_c)$ returns the number n_f of failures associated with t'_c .
$fails(t'_c)$	The function $fails(t'_c)$ returns 1 if $failures(t'_c) \geq 1$, and 0 otherwise.
$rank(t'_c)$	The function $rank(t'_c)$ returns the position of t'_c in a prioritized set T'_t .
$prec(t'_{c_1}, t'_{c_2})$	The function $prec(t'_{c_1}, t'_{c_2})$ returns 1 if $rank(t'_{c_1}) < rank(t'_{c_2})$.

idea is to evaluate whether failing test cases, with a greater number of failures, are ranked in the first positions in T'_t . To this end, a test case t'_c that does not fail and precedes failing test cases is penalized by their early scheduling.

$$TimeRank(t'_c) = \frac{|T'^{fail}| - [\neg(fails(t'_c)) \times \sum_{i=1}^{|T'^{fail}|} prec(t'_c, t'_{c_i})]}{|T'^{fail}|}, \quad (3)$$

where T'^{fail} is composed by the test cases of T'_t that failed. A non failed test case receives a reward given by the accumulated number of test cases which failed until its position in the prioritization rank, that is, it receives a reward decreased by the number of failing test cases ranked after it in the rank.

5 EVALUATION DESCRIPTION

The main hypothesis of this work is that MAB can be used to address the TCPPI problem in a very cost-effective way. Then, our experiment evaluates the COLEMAN applicability and performance in CI environments. We also perform a comparison with related work. The experiment is guided by the following research questions:

RQ1: *What is the best configuration for COLEMAN?* This question aims to identify the best MAB policy and reward function to be used with COLEMAN.

RQ2: *Is COLEMAN applicable in the CI development context?* This question is specially important for software testers that want to use COLEMAN in practice. It investigates whether the time spent in the prioritization is acceptable considering CI Cycles (commits).

RQ3: *Can COLEMAN outperform RETECS?* This question compares our approach, with RETECS, the RL approach, which is the most similar to COLEMAN.

To answer RQ1, we compare five MAB policies: Random, ϵ -Greedy, Greedy, UCB, and FRRMAB, (Section 4.1) and both reward functions: $RNFail$ and $TimeRank$ (Section 4.2.1). We use indicators commonly applied in TCP (Section 5.1). We evaluate three time constraints (budgets) considering: 10, 50, and 80 percent of the execution time of the overall test set available in each commit. The best policy identified in RQ1 is used to answer the remaining questions.

To answer RQ2, we compare the prioritization time spent by COLEMAN and the time between commits, as well as the percentage of reduced time to test execution.

To answer RQ3, we execute the implementation of RETECS available in the literature² by using ANN, which obtained the best results in comparison with a Tableau representation [14]. Our approach considers only a minimal information to prioritize the test cases, historical failure data, whilst RETECS needs additional information concerning each test case: duration, the time it was last executed, and results from its previous execution (passed or failed).

In this experiment, if a test case is removed in a commit, it is then removed along with its history. The results are obtained from 30 independent executions for each system, reward function, and time budget. All the experiments are performed on an Intel Xeon E5-2640 v3 with 2.60 GHz CPU, 94 GB RAM, running Linux Ubuntu 18.04.1 LTS.

5.1 Quality Indicators

To evaluate the performance of the approaches concerning failure detection effectiveness of a test suite T , we use NAPFD [32] (Equation 4) and APFDc [33] (Equation 5). Both metrics are extensions of the Average Percentage of Faults Detected (APFD) [34]. APFD indicates how quickly a set of prioritized test cases (T') can detect the faults present in the application being tested, and its value is calculated from the weighted average of the percentage of detected faults. APFD values range from zero to one. Higher values indicate that the faults are detected faster using fewer test cases. To calculate NAPFD, we consider the ratio between detected and detectable faults within T . This metric is adequate for prioritization of test cases when not all of them are executed, and some faults can be undetected.

$$NAPFD(T'_t) = p - \frac{\sum_{i=1}^n rank(T'_t)}{m \times n} + \frac{p}{2n}, \quad (4)$$

where m is the number of faults detected by all test cases; $rank(T'_t)$ is the position of T'_t in T' , if T'_t did not reveal a fault we set $T'_t = 0$; n is the number of tests cases in T' ; and p is the number of faults detected by T' divided by m . The last part of the equation represents the full area under the curve when the percentage of faults found is plotted on the y -axis and the percentage of the run test cases is on the x -axis. NAPFD is equal to APFD metric if all faults are detected.

The APFDc metric assumes that the test cases do not have the same cost, that is, some may be more costly to execute than others, maybe due to the fault severity or running time. The test cases are usually prioritized until a maximum cost is reached, which is feasible to execute. Furthermore, if both fault severity and test case costs are identical, APFDc can be used to compute the APFD value. In this work, we consider that all faults have same severity.

$$APFDc(T'_t) = \frac{\sum_{i=1}^m (\sum_{j=TF_i}^n c_j - 0.5c_{TF_i})}{\sum_{j=1}^n c_j \times m}, \quad (5)$$

where c_i is the cost of a test case T_i , and TF_i is the first test case from T' that reveals fault i .

2. <https://bitbucket.org/helges/RETECS>

TABLE 2
Test Case Set Information

Name	Period	Builds	Failures	Test Cases	Duration (min)	Interval (min)
Druid	2016/04/24-2016/11/08	286 (168)	270 (71)	2391 (1778-1910)	4.27 ± 10.66	384.76 ± 468.86
Fastjson	2016/04/15-2018/12/04	2710 (2371)	940 (323)	2416 (900-2102)	1.97 ± 0.89	233.22 ± 401.26
Deeplearning4j	2014/02/22-2016/01/01	3410 (483)	777 (323)	117 (1-52)	12.33 ± 14.91	306.05 ± 442.55
DSpace	2013/10/16-2019/01/08	6309 (5673)	13413 (387)	211 (16-136)	11.78 ± 7.03	291.29 ± 411.19
Guava	2014/11/06-2018/12/02	2011 (1689)	7659 (112)	568 (308-512)	62.53 ± 80.31	435.55 ± 464.52
OkHttp	2013/03/26-2018/05/30	9919 (6215)	9586 (1408)	289 (2-75)	7.64 ± 5.64	220.17 ± 405.93
Retrofit	2013/02/17-2018/11/26	3719 (2711)	611 (125)	206 (5-75)	2.40 ± 1.60	270.86 ± 449.41
ZXing	2014/01/17-2017/04/16	961 (605)	68 (11)	124 (81-123)	13.14 ± 12.37	411.10 ± 465.53
IOF/ROL	2015/02/13-2016/10/25	2392 (2392)	9289 (1627)	1941 (1-707)	1537.27 ± 2018.73	1324.26 ± 291.75
Paint Control	2016/01/12-2016/12/20	20711 (20711)	4956 (1980)	1980 (1-74)	424.46 ± 275.90	1417.86 ± 144.97
GSDTSR	2016/01/02-2016/02/01	259388 (259388)	3208 (2924)	5555 (1-390)	974.25 ± 4850.66	1439.91 ± 2.58

To evaluate the test suite efficiency concerning how fast it is to detect a fault, we use the Rank of the Failing Test Cases (RFTC). In this rank, lower values represent a faster failure detection. In this sense, we extract the order of the first test case that fails from the prioritized test suite. This metric is useful when we need fast feedback from test cases and there is only one fault that the test cases are seeking.

Furthermore, we define a metric (Equation (6)) named Normalized Time Reduction (NTR), in order to observe the difference between time spent until the first test case fails r_t and the total time spent to execute all tests $\hat{r}_t$. In this metric, only the commits that failed, CI^{fail} , are considered. In this way, we can evaluate the capability of an algorithm to reduce the time spent in a CI cycle.

$$NTR(\mathcal{A}) = \frac{\sum_{t=1}^{CI^{fail}} (\hat{r}_t - r_t)}{\sum_{t=1}^{CI^{fail}} (\hat{r}_t)}. \quad (6)$$

The last indicator used is Prioritization Time. Such measure computes the time spent (in seconds) by an algorithm to perform the prioritization. This value helps to observe whether an approach spends much time, what can make it impracticable for real scenarios.

We apply Kruskal-Wallis [35], Mann-Whitney [36], and Friedman [37] statistical tests with a confidence level of 95 percent. We use Kruskal-Wallis to evaluate the performance of the approaches in each system over 30 independent runs. We use Mann-Whitney to evaluate a pair of performances in the same system or for post-hoc analysis. We use Friedman to evaluate the approach behavior across different systems. To this end, each system becomes a dependent variable, in which we apply multiple approaches.

Additionally, to calculate the effect size magnitude of the difference between two groups, we use the Vargha and Delaney's $\hat{A}_{12}$ [38] metric. This measure ranges from 0 to 1 and defines the probability of a value, taken randomly from the first sample, is higher than a value taken randomly from the second sample. A *Negligible* magnitude ($\hat{A}_{12} < 0.56$) represents a very small difference among the values and usually does not yield statistical difference. The *Small* ($0.56 \leq \hat{A}_{12} < 0.64$) and *Medium* ($0.64 \leq \hat{A}_{12} < 0.71$) magnitudes represent small and medium differences among the values, and may or not yield statistical differences. Finally, a *Large* magnitude ($0.71 \leq \hat{A}_{12}$) represents a significantly

large difference that usually can be seen in the numbers without much effort.

5.2 Systems Under Test

We select non-toy, non-fork, and active GitHub (GH) projects considering *watchers* and *stars* on GH, as well as some systems already used in the literature [4], [14]. Most of them using Travis CI [39] and Maven.³ To collect the Travis CI build history, we adapt and use TravisTorrent⁴ [40].

The systems under test (SUTs) are detailed in Table 2. The second column shows the period of build logs analyzed. The third column presents the total of builds identified, and in parenthesis the number of builds included in the analysis. We discard build logs with some problem, identified by Travis CI, such as the ones related to non-valid build logs and test cases that did not execute. The fourth column shows the total of failures found, and in parenthesis the number of builds in which at least one test failed. The fifth column shows the number of different (unique) test cases identified from build logs, and in parenthesis the range of test cases executed in the builds. The last columns present, for each system, the mean ($\pm$ standard deviation) duration in minutes of the CI Cycles and the interval between them.

The systems IOF/ROL, Paint Control, and GSDTSR are selected for comparison because they are the same systems used in related work to evaluate RETECS [14]. However, in the related work the datasets are analyzed considering that a CI Cycle includes all the test cases executed per day. In our study, we consider the CI Cycle as a commit. In this way, we change the datasets representations to consider each date in the last run information is a commit. These systems have different characteristics (number of faults, test cases, and commits), and they can ensure the evaluation of the approaches in relation to the generalization capacity.

5.3 Parameters Setting

RETECS is available online and it is evaluated using ANN with the default values defined in related work [14], for

3. Maven is a build automation tool used primarily for Java projects. We choose projects which use Maven as a testing framework because it provides detailed output traces (more verbose).

4. <https://github.com/jacksonpradolima/travistorrent-tools>

TABLE 3
NAPFD and APFDc Values (Mean and Standard Deviation) for MAB Policies Using Time Budget of 50 percent

SUT	NAPFD					APFDc				
	FRRMAB	UCB	ϵ -Greedy	Greedy	Random	FRRMAB	UCB	ϵ -Greedy	Greedy	Random
RNFail - REWARD BASED ON FAILURES										
Druid	0.9333 $\pm$ 0.013 Δ	0.9422 $\pm$ 0.007 $\star$	0.8472 $\pm$ 0.110 Δ	0.8965 $\pm$ 0.072 ∇	0.7464 $\pm$ 0.014 Δ	0.9486 $\pm$ 0.016 $\star$	0.9486 $\pm$ 0.007 ∇	0.8477 $\pm$ 0.110 Δ	0.8979 $\pm$ 0.073 Δ	0.7506 $\pm$ 0.014 Δ
Fastjson	0.9174 $\pm$ 0.021 Δ	0.9597 $\pm$ 0.001 $\star$	0.9501 $\pm$ 0.005 Δ	0.9507 $\pm$ 0.005 Δ	0.9176 $\pm$ 0.003 Δ	0.9186 $\pm$ 0.021 Δ	0.9595 $\pm$ 0.001 $\star$	0.9491 $\pm$ 0.006 Δ	0.9498 $\pm$ 0.005 Δ	0.9193 $\pm$ 0.002 Δ
DeepLearning4j	0.7890 $\pm$ 0.001 Δ	0.7911 $\pm$ 0.002 Δ	0.8066 $\pm$ 0.003 Δ	0.8084 $\pm$ 0.002 $\star$	0.6381 $\pm$ 0.011 Δ	0.8106 $\pm$ 0.001 $\star$	0.7971 $\pm$ 0.002 Δ	0.7974 $\pm$ 0.004 Δ	0.7961 $\pm$ 0.003 Δ	0.6404 $\pm$ 0.016 Δ
DSpace	0.9724 $\pm$ 0.009 $\star$	0.9720 $\pm$ 0.001 ∇	0.9692 $\pm$ 0.002 ∇	0.9685 $\pm$ 0.002 ∇	0.9581 $\pm$ 0.001 Δ	0.9737 $\pm$ 0.009 $\star$	0.9730 $\pm$ 0.001 ∇	0.9713 $\pm$ 0.002 ∇	0.9704 $\pm$ 0.002 ∇	0.9587 $\pm$ 0.001 Δ
Guava	0.9653 $\pm$ 0.004 Δ	0.9750 $\pm$ 0.002 Δ	0.9768 $\pm$ 0.006 $\star$	0.9761 $\pm$ 0.010 ∇	0.9603 $\pm$ 0.002 Δ	0.9687 $\pm$ 0.003 Δ	0.9756 $\pm$ 0.002 ∇	0.9770 $\pm$ 0.006 $\star$	0.9758 $\pm$ 0.010 ∇	0.9611 $\pm$ 0.002 Δ
OkHttp	0.9192 $\pm$ 0.000 $\star$	0.9023 $\pm$ 0.001 Δ	0.9041 $\pm$ 0.005 Δ	0.8993 $\pm$ 0.006 Δ	0.8513 $\pm$ 0.002 Δ	0.9177 $\pm$ 0.000 $\star$	0.9039 $\pm$ 0.002 Δ	0.8994 $\pm$ 0.005 Δ	0.8950 $\pm$ 0.006 Δ	0.8549 $\pm$ 0.002 Δ
Retrolit	0.9853 $\pm$ 0.000 $\star$	0.9799 $\pm$ 0.001 Δ	0.9717 $\pm$ 0.001 Δ	0.9717 $\pm$ 0.001 Δ	0.9710 $\pm$ 0.002 Δ	0.9850 $\pm$ 0.000 $\star$	0.9794 $\pm$ 0.001 Δ	0.9722 $\pm$ 0.001 Δ	0.9722 $\pm$ 0.001 Δ	0.9712 $\pm$ 0.001 Δ
Zxing	0.9846 $\pm$ 0.000 Δ	0.9876 $\pm$ 0.000 Δ	0.9873 $\pm$ 0.002 Δ	0.9869 $\pm$ 0.002 Δ	0.9892 $\pm$ 0.002 $\star$	0.9862 $\pm$ 0.000 Δ	0.9877 $\pm$ 0.000 Δ	0.9876 $\pm$ 0.001 Δ	0.9873 $\pm$ 0.001 Δ	0.9893 $\pm$ 0.001 $\star$
IOF/ROL	0.5046 $\pm$ 0.002 $\star$	0.4791 $\pm$ 0.003 Δ	0.4792 $\pm$ 0.002 Δ	0.4790 $\pm$ 0.002 Δ	0.4786 $\pm$ 0.002 Δ	0.5081 $\pm$ 0.002 $\star$	0.4794 $\pm$ 0.003 Δ	0.4796 $\pm$ 0.002 Δ	0.4793 $\pm$ 0.002 Δ	0.4788 $\pm$ 0.002 Δ
Paint Control	0.9150 $\pm$ 0.000 $\star$	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9162 $\pm$ 0.000 $\star$	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ
GSDTSR	0.9893 $\pm$ 0.000 $\star$	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9894 $\pm$ 0.000 $\star$	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9892 $\pm$ 0.000 Δ
Average	0.8978	0.8993	0.8905	0.8946	0.8558	0.9021	0.9007	0.8895	0.8934	0.8571
TimeRank - TIME-RANKED REWARD										
Druid	0.9710 $\pm$ 0.008 $\star$	0.8422 $\pm$ 0.066 Δ	0.8767 $\pm$ 0.040 Δ	0.8503 $\pm$ 0.048 Δ	0.7489 $\pm$ 0.014 Δ	0.9787 $\pm$ 0.009 $\star$	0.8768 $\pm$ 0.078 Δ	0.9888 $\pm$ 0.042 Δ	0.8871 $\pm$ 0.056 Δ	0.7534 $\pm$ 0.014 Δ
Fastjson	0.9118 $\pm$ 0.028 Δ	0.9544 $\pm$ 0.002 $\star$	0.9455 $\pm$ 0.004 Δ	0.9240 $\pm$ 0.009 Δ	0.9181 $\pm$ 0.003 Δ	0.9140 $\pm$ 0.027 Δ	0.9565 $\pm$ 0.002 $\star$	0.9516 $\pm$ 0.004 Δ	0.9303 $\pm$ 0.009 Δ	0.9199 $\pm$ 0.003 Δ
DeepLearning4j	0.8200 $\pm$ 0.000 $\star$	0.7193 $\pm$ 0.002 Δ	0.7028 $\pm$ 0.004 Δ	0.7047 $\pm$ 0.002 Δ	0.6366 $\pm$ 0.007 Δ	0.8134 $\pm$ 0.001 $\star$	0.7879 $\pm$ 0.004 Δ	0.7774 $\pm$ 0.007 Δ	0.7805 $\pm$ 0.004 Δ	0.6405 $\pm$ 0.012 Δ
DSpace	0.9766 $\pm$ 0.008 $\star$	0.9659 $\pm$ 0.001 Δ	0.9606 $\pm$ 0.002 Δ	0.9602 $\pm$ 0.002 Δ	0.9582 $\pm$ 0.001 Δ	0.9767 $\pm$ 0.009 $\star$	0.9683 $\pm$ 0.001 Δ	0.9646 $\pm$ 0.002 Δ	0.9649 $\pm$ 0.002 Δ	0.9588 $\pm$ 0.001 Δ
Guava	0.9675 $\pm$ 0.007 Δ	0.9698 $\pm$ 0.002 Δ	0.9738 $\pm$ 0.002 $\star$	0.9734 $\pm$ 0.004 ∇	0.9608 $\pm$ 0.002 Δ	0.9672 $\pm$ 0.007 Δ	0.9740 $\pm$ 0.003 Δ	0.9786 $\pm$ 0.003 Δ	0.9824 $\pm$ 0.004 $\star$	0.9614 $\pm$ 0.002 Δ
OkHttp	0.9317 $\pm$ 0.000 $\star$	0.8753 $\pm$ 0.001 Δ	0.8725 $\pm$ 0.002 Δ	0.8677 $\pm$ 0.004 Δ	0.8514 $\pm$ 0.002 Δ	0.9246 $\pm$ 0.000 $\star$	0.8930 $\pm$ 0.001 Δ	0.8881 $\pm$ 0.002 Δ	0.8866 $\pm$ 0.004 Δ	0.8550 $\pm$ 0.002 Δ
Retrolit	0.9893 $\pm$ 0.000 $\star$	0.9789 $\pm$ 0.001 Δ	0.9715 $\pm$ 0.002 Δ	0.9689 $\pm$ 0.001 Δ	0.9707 $\pm$ 0.001 Δ	0.9885 $\pm$ 0.000 $\star$	0.9798 $\pm$ 0.001 Δ	0.9733 $\pm$ 0.002 Δ	0.9712 $\pm$ 0.001 Δ	0.9706 $\pm$ 0.001 Δ
Zxing	0.9857 $\pm$ 0.000 Δ	0.9879 $\pm$ 0.001 Δ	0.9882 $\pm$ 0.002 ∇	0.9861 $\pm$ 0.001 Δ	0.9891 $\pm$ 0.001 $\star$	0.9869 $\pm$ 0.000 Δ	0.9880 $\pm$ 0.001 Δ	0.9882 $\pm$ 0.002 Δ	0.9861 $\pm$ 0.001 Δ	0.9894 $\pm$ 0.001 $\star$
IOF/ROL	0.5189 $\pm$ 0.002 $\star$	0.4787 $\pm$ 0.002 Δ	0.4789 $\pm$ 0.002 Δ	0.4786 $\pm$ 0.002 Δ	0.4785 $\pm$ 0.002 Δ	0.5223 $\pm$ 0.002 $\star$	0.4789 $\pm$ 0.002 Δ	0.4792 $\pm$ 0.002 Δ	0.4789 $\pm$ 0.002 Δ	0.4786 $\pm$ 0.002 Δ
Paint Control	0.9150 $\pm$ 0.000 $\star$	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9162 $\pm$ 0.000 $\star$	0.9146 $\pm$ 0.000 Δ	0.9146 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ	0.9145 $\pm$ 0.000 Δ
GSDTSR	0.9894 $\pm$ 0.000 $\star$	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9894 $\pm$ 0.000 $\star$	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ	0.9891 $\pm$ 0.000 Δ
Average	0.9070	0.8796	0.8795	0.8743	0.8560	0.9071	0.8915	0.8912	0.8883	0.8574

This table reports the NAPFD and APFDc results (averages $\pm$ standard deviation) obtained from 30 independent runs with time budget of 50 percent and organized by each Reward Function under study (see Section 4.2.1). Values highlighted in bold with a “ $\star$ ” symbol denotes the best algorithm for a Reward Function in a SUT and, in gray, results that are statistically equal to the best one. A “ ∇ ” indicates that the effect size was negligible in relation to the best, while “ Δ ” denotes a small magnitude, “ Δ ” a medium magnitude, and “ Δ ” a large magnitude. The effect size was performed during the post-hoc tests, that is, when there is a statistical difference.

Hidden Nodes, Replay Memory, and Replay Batch Size, with, respectively, 12, 10000, and 1000. In this way, a tuning phase was necessary only to choose the parameters values for the MAB policies UCB, FRRMAB, and ϵ -Greedy. They are the scaling factor C to control the trade-off among EvE for UCB and FRRMAB policies, the sliding window size W and decayed factor DF for FRRMAB, and the probability ϵ for ϵ -Greedy policy. The possible values used in this phase for C are 0.3, 0.5, 1.0, and 2.0, for W were 10, 50, 100, 150, 200, 250, and 300, and for ϵ 0.1, 0.2, 0.5, 0.7, and 0.9. After some empirical studies, the decayed factor is no longer used and a default value equals to 1 is defined.

We conduct an empirical evaluation with 10 independent runs for each MAB policy with different parameters. For this evaluation, we consider the systems DeepLearning4j and Fastjson. These systems are chosen because they have the best trade-off between the mean number of failures by CI Cycles and the number of CI Cycles. Additionally, we define a test budget for a CI Cycle with a fixed percentage of 50 percent of the required time as adopted in [14]. The best parameters found for the MAB policies are: ϵ -Greedy and UCB with 0.5 (in the TimeRank function) and 0.3 (RNFail) for parameter C ; and FRRMAB with 0.3 for C and 100 for W .

5.4 Threats to Validity

We identify the following points that can be threats to the validity of the results. The first threat is the parameter configuration of the algorithms. Other parameters can lead to different results. To mitigate this threat, we empirically evaluate different ranges of parameters for the MAB policies and for RETECS we adopt the configuration of related work, since we are using the same systems.

The dataset representation used is a threat. We change information about the CI Cycles for systems IOF/ROL, Paint Control, and GSDTSR. This change can impact the results, mainly because we do not know with precision

what information is from each commit and whether RETECS is deeply dependent on this kind of representation. For this reason, we group the data by the last run date.

6 RESULTS AND ANALYSES

In this section, the experimental results are presented and analyzed aiming to answer the posed questions. Supplementary material with datasets, results, and additional analysis can be found in our public repository [21].

6.1 RQ1: COLEMAN Configuration

As mentioned before, to answer RQ1, we compare the five MAB policies with both reward functions taking account three budgets. We use the indicators: NAPFD, APFDc, and RFTC. The results and a complete analysis are available in supplementary material [21]. Due to lack of space, we detail only NAPFD and APFDc results regarding the budget of 50 percent (Table 3). According to Spieker *et al.* [14], a time budget of 50 percent presents a constraint that allows better comparison whilst keeps the difficulty inherent from the problem.

The average is computed using results from 30 independent executions found by each policy in each SUT. Values highlighted in bold are the best, and values that are statistically equivalent to the best ones have their corresponding cells painted in light gray. Furthermore, we use different symbols to indicate the effect size magnitude concerning the best values. For each comparison in each SUT, we use the Kruskal-Wallis test. When detected statistical difference, we apply a post-hoc analysis using the Mann-Whitney test with Bonferroni p -value adjustment method to find the statistical difference among the policies. In order to evaluate the performance of the policy concerning all systems, we compute the average across the systems. The average values

obtained are shown in the last line in each table. We compare these values using Friedman.

Regarding NAPFD, Table 3 shows FRRMAB stands out in 68 percent of the cases (15 out of 22 cases, considering 11 systems and 2 functions). Such cases are represented by grey cells in the table. FRRMAB is followed by UCB with 50 percent of the cases (11 out of 22). FRRMAB is the best with statistical difference over the others in 8 cases. UCB is the best only in one case for *RNFail*. The effect size results endorse what we observed in the numbers. Besides that, when there is a statistical difference among policies, the effect size tends to be *large*. The performance of these policies is similar for both functions. For *RNFail*, FRRMAB stands out in 7 cases (out of 11), and UCB stands out in 5 (out of 11). For *TimeRank*, FRRMAB stands out in 8, and UCB stands out in 6. Overall, we observe that for *RNFail* the other MAB policies perform better. For this reward function, more policies have values that are close to the best one. Among the MAB policies, the Random policy has the worst performance. Regarding APFDc we have similar findings. FRRMAB stands out in 72 percent of the cases (16 out of 22), and UCB in 45 percent of the cases (10 out of 22).

For the other budgets of 10 and 80 percent, we observed similar results regarding the performance of the policies and functions. As expected, the values for all policies are better for the budget of 80 percent, which is less restrictive. Considering NAPFD and all the 33 cases involving the three budgets (11 systems $\times$ 3 time budgets), FRRMAB stands out the other policies in ≈ 76 percent of the cases (25 out of 33) and 82 percent (27 cases), respectively, for the *RNFail* and *TimeRank* functions. FRRMAB is the best policy with statistical difference in ≈ 40 percent of the cases (13 out of 33) for both reward functions, and UCB policy is the best in ≈ 6 percent of the cases (2) and ≈ 3 percent of the cases (1), respectively, for the *RNFail* and *TimeRank* functions. FRRMAB also obtains the best APFDc values and stands out in ≈ 82 percent of the cases (27 out of 33) for the *RNFail* function and in ≈ 76 percent of the cases (25) for the *TimeRank* function.

For *IOF/ROL*, the approach had the worst performance. This system has a high test case volatility and a high number of peaks of failure detection. In this way, it is an example of that to find a solution to the TCP CI problem is hard. We observe that the Random policy is defeated by MAB policies in almost all systems, except in *ZXing* in both reward functions. To understand this behavior, we analyze the failures detected over the cycles. In this system, we verify peaks in the failure detection in a few commits and long periods without failures. This scenario endorses our approach behavior once that it is based on historical test data. Furthermore, this system has a low test case volatility.

RFTC results are available in supplementary material [21]. We observe that the NAPFD average values found in each independent execution by reward functions and the early fault detection given by RFTC are, in most of the cases, correlated, that is, good NAPFD values provide good RFTC values. In most of the systems, the MAB policies are better than Random policy, as well as they are more stable (low dispersion of values). Among the MAB policies, FRRMAB is more stable than the other ones. This shows that the use of a sliding window is interesting when a system contains

TABLE 4
Mean and Standard Deviation Prioritization Time (in seconds)
With Time Budget of 50 percent

SUT	PRIORITIZATION TIME (SEC.)			
	FRRMAB		ANN	
	RNFail	TimeRank	RNFail	TimeRank
Druid	0.2474 $\pm$ 0.040	0.2373 $\pm$ 0.041	0.3881 $\pm$ 0.268	0.3844 $\pm$ 0.279
Fastjson	0.3916 $\pm$ 0.191	0.3879 $\pm$ 0.191	0.2474 $\pm$ 1.382	0.2395 $\pm$ 1.288
Deeplearning4j	0.0271 $\pm$ 0.002	0.0271 $\pm$ 0.002	0.0038 $\pm$ 0.006	0.0187 $\pm$ 0.107
DSpace	0.0287 $\pm$ 0.002	0.0287 $\pm$ 0.002	0.0101 $\pm$ 0.042	0.0507 $\pm$ 0.186
Guava	0.0609 $\pm$ 0.016	0.0609 $\pm$ 0.016	0.0335 $\pm$ 0.039	0.0405 $\pm$ 0.066
OkHttp	0.0283 $\pm$ 0.002	0.0283 $\pm$ 0.002	0.0079 $\pm$ 0.022	0.0371 $\pm$ 0.125
Retrofit	0.0277 $\pm$ 0.002	0.0277 $\pm$ 0.002	0.0050 $\pm$ 0.010	0.0178 $\pm$ 0.079
ZXing	0.0343 $\pm$ 0.003	0.0343 $\pm$ 0.003	0.0156 $\pm$ 0.025	0.0229 $\pm$ 0.069
IOF/ROL	0.0278 $\pm$ 0.005	0.0278 $\pm$ 0.005	0.0034 $\pm$ 0.009	0.5364 $\pm$ 1.221
Paint Control	0.0256 $\pm$ 0.002	0.0256 $\pm$ 0.002	0.0020 $\pm$ 0.005	0.0028 $\pm$ 0.028
GSDTSR	0.0253 $\pm$ 0.002	0.0253 $\pm$ 0.002	0.0027 $\pm$ 0.014	0.0028 $\pm$ 0.013
Average	0.0841	0.0828	0.0654	0.1231

peaks of detecting faults and periods of stability, as well as when there is a large number of commits.

RQ1: Results show FRRMAB is the best MAB policy, regarding NAPFD, APFDc values, and early fault detection given by the indicator RFTC. This happens for both reward functions, and the three budgets evaluated. As expected, better indicator values are obtained for the less restrictive budget of 80 percent. This happens for all policies. In particular, the combination of FRRMAB with *TimeRank* function provides better performance.

6.2 RQ2: COLEMAN Applicability

To answer RQ2, we use Table 4. The second and third columns presents, respectively, the mean prioritization time and standard deviation of COLEMAN with the FRRMAB policy (the best policy found in the analysis of RQ1), and time budget of 50 percent. In such a table, we can observe the time spent is negligible in most of the systems. A great time is spent in *Druid* and *Fastjson*, systems that also have a great number of test cases in a CI Cycle. If we take the standard deviation and the worst case (*Fastjson*), COLEMAN takes less than one second to execute. The mean time spent with both reward functions is similar, as well as we do not observe any impact on the other budgets.

However, we need to consider the impact of this time in the complete CI Cycle and the interval between the CI Cycles, and consequently, to observe whether the spent prioritization time is expensive. In addition, it is important to analyze the impact concerning the time spent to reveal the first failure, which can be used to reduce the time spent in a CI Cycle once that the test execution cost can be reduced when a failure is revealed because the test is ended. For this end, Table 5 presents the time reduction average, given by the indicator NTR (Equation 6) with both reward functions, over the three budgets. Refer to Table 2 to see duration of the CI cycles and the interval between them.

Table 5 shows percentages of time reduction provided by our approach are close to 73 percent for *IOF/ROL*, 47 percent for *Deeplearning4j*, and 43 percent for *Druid*. This reduction in a CI cycle is due to the early fault detection provided by our approach. But, in most systems, we can see a low percentage. The performance of both functions is similar, as well as the NTR values for each system. In overall, the *RNFail* function obtains better reduction than *TimeRank*

TABLE 5
Mean and Standard Deviation NAPFD, APFDc, and NTR Values: COLEMAN Against RETECS

SUT	NAPFD				APFDc				NTR			
	RNFAIL		TIMERANK		RNFAIL		TIMERANK		RNFAIL		TIMERANK	
	ANN	FRRMAB	ANN	FRRMAB	ANN	FRRMAB	ANN	FRRMAB	ANN	FRRMAB	ANN	FRRMAB
TIME BUDGET: 10%												
Druid	0.6768 ± 0.129 ▲	0.6801 ± 0.052 ★	0.6488 ± 0.074 ▲	0.7137 ± 0.074 ★	0.8067 ± 0.088 ★	0.6944 ± 0.063 ▲	0.7815 ± 0.051 ★	0.7181 ± 0.076 ▲	0.1863 ± 0.124	0.2027 ± 0.115	0.1556 ± 0.077	0.2355 ± 0.135
Fastjson	0.8713 ± 0.015 ▲	0.9030 ± 0.014 ★	0.8719 ± 0.005 ▲	0.8980 ± 0.018 ★	0.8805 ± 0.014 ▲	0.9064 ± 0.013 ★	0.8810 ± 0.014 ▲	0.8974 ± 0.018 ★	0.0194 ± 0.014	0.3117 ± 0.021	0.0219 ± 0.007	0.2674 ± 0.232
DeepLearning4j	0.6615 ± 0.072 ▲	0.7333 ± 0.002 ★	0.6739 ± 0.016 ▲	0.7716 ± 0.000 ★	0.8135 ± 0.023 ★	0.7766 ± 0.001 ▲	0.8185 ± 0.010 ★	0.7773 ± 0.001 ▲	0.5488 ± 0.029	0.4626 ± 0.001	0.5546 ± 0.015	0.4463 ± 0.010
DSpace	0.9437 ± 0.001 ▲	0.9489 ± 0.003 ★	0.9410 ± 0.001 ▲	0.9496 ± 0.004 ★	0.9480 ± 0.001 ▲	0.9510 ± 0.002 ★	0.9458 ± 0.001 ▲	0.9513 ± 0.004 ★	0.0188 ± 0.001	0.0370 ± 0.015	0.0105 ± 0.001	0.0393 ± 0.018
Guava	0.9676 ± 0.015 ★	0.9554 ± 0.002 ▲	0.9563 ± 0.004	0.9586 ± 0.001 ★	0.9811 ± 0.007 ★	0.9561 ± 0.001 ▲	0.9761 ± 0.003 ★	0.9582 ± 0.001 ▲	0.0449 ± 0.015	0.0471 ± 0.016	0.0367 ± 0.005	0.0492 ± 0.015
OkHttp	0.8357 ± 0.002 ★	0.8323 ± 0.001 ▲	0.8095 ± 0.006 ▲	0.8407 ± 0.000 ★	0.8484 ± 0.001 ★	0.8378 ± 0.000 ▲	0.8292 ± 0.006 ▲	0.8425 ± 0.000 ★	0.0830 ± 0.001	0.0658 ± 0.000	0.0579 ± 0.008	0.0702 ± 0.000
Retrofit	0.9641 ± 0.001 ★	0.9639 ± 0.000	0.9621 ± 0.001 ▲	0.9642 ± 0.000 ★	0.9672 ± 0.001 ★	0.9646 ± 0.000 ▲	0.9655 ± 0.001 ★	0.9648 ± 0.000 ▲	0.0088 ± 0.000	0.0070 ± 0.000	0.0076 ± 0.001	0.0073 ± 0.000
XZing	0.9854 ± 0.000 ★	0.9826 ± 0.000 ▲	0.9855 ± 0.000 ★	0.9828 ± 0.000 ▲	0.9893 ± 0.000 ★	0.9832 ± 0.000 ▲	0.9893 ± 0.000 ★	0.9835 ± 0.000 ▲	0.0122 ± 0.000	0.0037 ± 0.000	0.0126 ± 0.001	0.0037 ± 0.000
IOF/ROL	0.3704 ± 0.005 ★	0.3632 ± 0.001 ▲	0.3779 ± 0.003 ★	0.3670 ± 0.001 ▲	0.3746 ± 0.005 ★	0.3661 ± 0.000 ▲	0.3819 ± 0.003 ★	0.3701 ± 0.001 ▲	0.5133 ± 0.030	0.5855 ± 0.007	0.5646 ± 0.015	0.5701 ± 0.004
Paint Control	0.9078 ± 0.000 ★	0.9076 ± 0.000 ▲	0.9077 ± 0.000 ★	0.9076 ± 0.000 ▲	0.9082 ± 0.000 ★	0.9080 ± 0.000 ▲	0.9081 ± 0.000 ★	0.9080 ± 0.000 ▲	0.1151 ± 0.000	0.1133 ± 0.000	0.1139 ± 0.000	0.1131 ± 0.000
GSDTSR	0.9893 ± 0.000 ★	0.9894 ± 0.000 ★	0.9893 ± 0.000 ▲	0.9894 ± 0.000 ★	0.9894 ± 0.000	0.9894 ± 0.000	0.9894 ± 0.000	0.9894 ± 0.000	0.0087 ± 0.000	0.0093 ± 0.000	0.0090 ± 0.000	0.0096 ± 0.000
Average	0.8340	0.8436	0.8294	0.8494	0.8643	0.8487	0.8294	0.8494	0.1428	0.1653	0.1404	0.1665
TIME BUDGET: 50%												
Druid	0.6851 ± 0.134 ▲	0.9333 ± 0.013 ★	0.6323 ± 0.074 ▲	0.9710 ± 0.008 ★	0.8147 ± 0.102 ▲	0.9486 ± 0.016 ★	0.7597 ± 0.051 ▲	0.9787 ± 0.009 ★	0.1840 ± 0.137	0.4057 ± 0.013	0.1213 ± 0.071	0.4225 ± 0.008
Fastjson	0.8714 ± 0.007 ▲	0.9174 ± 0.002 ★	0.8902 ± 0.013 ▲	0.9118 ± 0.028 ★	0.9326 ± 0.005 ★	0.9186 ± 0.021 ▲	0.9392 ± 0.017 ★	0.9140 ± 0.027 ▲	0.0399 ± 0.010	0.3339 ± 0.262	0.0602 ± 0.020	0.3394 ± 0.282
DeepLearning4j	0.7049 ± 0.070 ▲	0.7890 ± 0.001 ★	0.6562 ± 0.018 ▲	0.8200 ± 0.000 ★	0.8331 ± 0.041 ★	0.8106 ± 0.001 ▲	0.8379 ± 0.012 ★	0.8134 ± 0.001 ▲	0.5276 ± 0.039	0.4695 ± 0.000	0.5447 ± 0.010	0.4625 ± 0.000
DSpace	0.9568 ± 0.001 ▲	0.9724 ± 0.009 ★	0.9485 ± 0.001 ▲	0.9766 ± 0.008 ★	0.9835 ± 0.001 ★	0.9737 ± 0.009 ★	0.9615 ± 0.001 ▲	0.9767 ± 0.009 ★	0.0334 ± 0.001	0.0751 ± 0.031	0.0218 ± 0.002	0.0776 ± 0.034
Guava	0.9502 ± 0.015 ▲	0.9653 ± 0.003 ★	0.9474 ± 0.004 ★	0.9675 ± 0.007 ★	0.9767 ± 0.008 ★	0.9673 ± 0.003 ▲	0.9806 ± 0.005 ★	0.9672 ± 0.007 ▲	0.0303 ± 0.016	0.0662 ± 0.024	0.0387 ± 0.006	0.0659 ± 0.020
OkHttp	0.8812 ± 0.010 ▲	0.9192 ± 0.000 ★	0.8446 ± 0.003 ▲	0.9317 ± 0.000 ★	0.8878 ± 0.015 ▲	0.9177 ± 0.000 ★	0.8869 ± 0.002 ▲	0.9246 ± 0.000 ★	0.1118 ± 0.014	0.1431 ± 0.000	0.1060 ± 0.003	0.1486 ± 0.000
Retrofit	0.9706 ± 0.002 ▲	0.9853 ± 0.000 ★	0.9718 ± 0.002 ▲	0.9893 ± 0.000 ★	0.9762 ± 0.002 ▲	0.9850 ± 0.000 ★	0.9778 ± 0.002 ▲	0.9885 ± 0.000 ★	0.0134 ± 0.001	0.0156 ± 0.000	0.0138 ± 0.001	0.0172 ± 0.000
XZing	0.9878 ± 0.000 ★	0.9846 ± 0.001 ▲	0.9881 ± 0.001 ★	0.9857 ± 0.001 ▲	0.9862 ± 0.001 ▲	0.9862 ± 0.001 ▲	0.9862 ± 0.001 ▲	0.9862 ± 0.001 ▲	0.0201 ± 0.000	0.0196 ± 0.000	0.0201 ± 0.000	0.0110 ± 0.000
IOF/ROL	0.5101 ± 0.007 ★	0.5046 ± 0.002 ▲	0.5025 ± 0.006 ▲	0.5189 ± 0.002 ★	0.5175 ± 0.008 ★	0.5081 ± 0.002 ▲	0.5043 ± 0.006 ▲	0.5223 ± 0.002 ★	0.7037 ± 0.019	0.7110 ± 0.003	0.6834 ± 0.014	0.7193 ± 0.003
Paint Control	0.9150 ± 0.000 ★	0.9150 ± 0.000 ★	0.9150 ± 0.000 ★	0.9150 ± 0.000 ★	0.9171 ± 0.000 ★	0.9162 ± 0.000 ★	0.9140 ± 0.000 ★	0.9162 ± 0.000 ★	0.1283 ± 0.000	0.1222 ± 0.000	0.1142 ± 0.001	0.1223 ± 0.000
GSDTSR	0.9911 ± 0.000 ★	0.9893 ± 0.000 ★	0.9906 ± 0.000 ★	0.9894 ± 0.000 ★	0.9911 ± 0.000 ★	0.9894 ± 0.000 ★	0.9910 ± 0.000 ★	0.9894 ± 0.000 ★	0.0199 ± 0.000	0.0093 ± 0.000	0.0179 ± 0.000	0.0096 ± 0.000
Average	0.8567	0.8978	0.8451	0.9070	0.8919	0.9021	0.8862	0.9071	0.1648	0.2166	0.1583	0.2178
TIME BUDGET: 80%												
Druid	0.6490 ± 0.113 ▲	0.9380 ± 0.012 ★	0.6551 ± 0.099 ▲	0.9830 ± 0.003 ★	0.7142 ± 0.111 ▲	0.9469 ± 0.015 ★	0.6881 ± 0.090 ▲	0.9912 ± 0.004 ★	0.1477 ± 0.113	0.4069 ± 0.014	0.1230 ± 0.096	0.4292 ± 0.004
Fastjson	0.8708 ± 0.007 ▲	0.9536 ± 0.010 ★	0.8925 ± 0.010 ▲	0.9242 ± 0.028 ★	0.9037 ± 0.008 ▲	0.9488 ± 0.012 ★	0.9133 ± 0.015 ▲	0.927 ± 0.026 ★	0.0385 ± 0.011	0.4613 ± 0.278	0.0516 ± 0.018	0.3963 ± 0.301
DeepLearning4j	0.7058 ± 0.091 ▲	0.8424 ± 0.001 ★	0.6640 ± 0.016 ▲	0.8641 ± 0.001 ★	0.8158 ± 0.064 ★	0.8065 ± 0.002 ▼	0.8522 ± 0.012 ★	0.7989 ± 0.001 ▲	0.5016 ± 0.058	0.4224 ± 0.001	0.5475 ± 0.007	0.4047 ± 0.000
DSpace	0.9601 ± 0.001 ▲	0.9792 ± 0.006 ★	0.9508 ± 0.001 ▲	0.9825 ± 0.007 ★	0.9738 ± 0.001 ▲	0.9796 ± 0.006 ★	0.9639 ± 0.001 ▲	0.9810 ± 0.008 ★	0.0374 ± 0.001	0.0800 ± 0.031	0.0269 ± 0.002	0.0812 ± 0.034
Guava	0.9441 ± 0.012 ▲	0.9784 ± 0.001 ★	0.9581 ± 0.007 ▲	0.9841 ± 0.014 ★	0.9842 ± 0.010 ▲	0.9780 ± 0.013 ▲	0.9849 ± 0.013 ▲	0.9812 ± 0.015 ★	0.0374 ± 0.001	0.0800 ± 0.031	0.0269 ± 0.002	0.0812 ± 0.034
OkHttp	0.9027 ± 0.013 ▲	0.9350 ± 0.001 ★	0.8558 ± 0.004 ▲	0.9478 ± 0.004 ★	0.9836 ± 0.020 ▲	0.9271 ± 0.000 ★	0.8974 ± 0.001 ▲	0.9362 ± 0.001 ★	0.1112 ± 0.017	0.0493 ± 0.000	0.1153 ± 0.003	0.1551 ± 0.000
Retrofit	0.9724 ± 0.005 ▲	0.9881 ± 0.000 ★	0.9745 ± 0.003 ▲	0.9916 ± 0.000 ★	0.9785 ± 0.004 ▲	0.9875 ± 0.000 ★	0.9808 ± 0.003 ▲	0.9903 ± 0.000 ★	0.0140 ± 0.001	0.0161 ± 0.000	0.0145 ± 0.001	0.0179 ± 0.000
XZing	0.9878 ± 0.000 ★	0.9972 ± 0.000 ★	0.9883 ± 0.001 ▲	0.9996 ± 0.000 ★	0.9953 ± 0.000 ★	0.9953 ± 0.000 ★	0.9953 ± 0.000 ★	0.9996 ± 0.000 ★	0.0201 ± 0.000	0.0224 ± 0.000	0.0197 ± 0.002	0.0227 ± 0.000
IOF/ROL	0.5495 ± 0.006 ▲	0.5369 ± 0.002 ★	0.5287 ± 0.007 ▲	0.5678 ± 0.001 ★	0.5593 ± 0.006 ★	0.5591 ± 0.002	0.5311 ± 0.006 ▲	0.5699 ± 0.001 ★	0.7263 ± 0.015	0.7295 ± 0.002	0.6857 ± 0.011	0.7334 ± 0.001
Paint Control	0.9162 ± 0.000 ★	0.9171 ± 0.000 ★	0.9160 ± 0.000 ★	0.9171 ± 0.000 ★	0.9187 ± 0.000 ★	0.9176 ± 0.000 ★	0.9178 ± 0.000 ★	0.9177 ± 0.000 ★	0.1285 ± 0.000	0.1289 ± 0.000	0.1161 ± 0.001	0.1204 ± 0.000
GSDTSR	0.9921 ± 0.000 ★	0.9893 ± 0.000 ★	0.9914 ± 0.000 ★	0.9894 ± 0.000 ★	0.9919 ± 0.000 ★	0.9894 ± 0.000 ★	0.9917 ± 0.000 ★	0.9894 ± 0.000 ★	0.0218 ± 0.001	0.0093 ± 0.000	0.0203 ± 0.000	0.0096 ± 0.000
Average	0.8591	0.9159	0.8523	0.9228	0.8816	0.9126	0.8817	0.9167	0.1611	0.2272	0.1596	0.2236

(See caption of Table 3 for a description of the headings.)

considering time budget of 80 percent, and the *TimeRank* is better in the other time budgets. Comparing the average percentage for the budget of 50 percent we observe an improvement regarding the budget of 10 percent. But this difference is slightly lower when we compare the budgets of 50 and 80 percent. Then, the budget may be a reason to explain low reduction. Other possible reasons are the difficulty inherent to the TCPPI problem and the need to consider other aspects in the prioritization, such as the time to execute each test case.

Regarding the time spent in a CI Cycle and interval between commits for each system, we observe that a new commit is typically performed after a CI Cycle is finished and with a considered time, due to the time between commits is, in most systems, higher than the time spent by a CI Cycle. In this way, the systems chosen do not present a situation with multiple test requests. In addition, the time spent in a CI Cycle and between cycles is in minutes, and as our approach spent, in the worst case, less than one second to prioritize the test cases, there is not a negative impact in the use of our approach.

RQ2: Our approach is applicable to the CI context. It contributes to reducing the time spent in a CI cycle, even with a restrictive budget of 10 percent. In some cases, the reduction can achieve a percentage of 70 percent.

6.3 RQ3: Comparing COLEMAN and RETECS

To answer RQ3, we compare COLEMAN using FRRMAB (the best policy according to RQ1) with RETECS using ANN. Table 5 shows the NAPFD and APFDc results obtained.

As we can observe for *RNFail* function and NAPFD, FRRMAB stands out in ≈ 70 percent of the cases (23 out of

33). ANN stands out in 33 percent (11 cases). FRRMAB is the best with statistical difference in ≈ 66 percent (22 out of 33) of the cases against 30 percent (10 cases) for ANN. Regarding the NAPFD average across the SUTs, FRRMAB obtained the best results, with statistical difference in the time budget of 80 percent. With the *TimeRank* function, FRRMAB presents even better results. FRRMAB stands out in ≈ 82 percent (27 out of 33) of the cases against 21 percent (7 cases) for ANN. FRRMAB is the best with statistical difference in ≈ 79 percent of the cases (26 out of 33) against 18 percent (6 cases) for ANN. Across the SUTs, FRRMAB is

TABLE 6
Mean and Standard Deviation RFTC Values:
COLEMAN Against RETECS

SUT	RFTC			
	RNFail		TimeRank	
	ANN	FRRMAB	ANN	FRRMAB
Time Budget: 10%				
Druid	1166.4411 ± 546.049 ▲	209.3289 ± 98.987 ★	1240.8713 ± 356.817 ▲	56.1579 ± 31.056 ★
Fastjson	1094.1147 ± 209.106 ▲	184.9911 ± 65.066 ★	998.7949 ± 177.738 ▲	96.0378 ± 36.222 ★
DeepLearning4j	5.3151 ± 2.456 ▲	2.3049 ± 0.048 ★	5.7919 ± 0.677 ▲	2.0437 ± 0.007 ★
DSpace	11.9561 ± 0.753 ▲	3.024 ± 1.33 ★	13.6276 ± 0.754 ▲	2.1428 ± 0.67 ★
Guava	129.3477 ± 99.443 ▲	31.8991 ± 20.546 ★	191.1219 ± 29.545 ▲	15.0977 ± 13.75 ★
OkHttp	7.4112 ± 0.397 ▲	4.3424 ± 0.0 ★	15.8935 ± 1.951 ▲	1.8039 ± 0.0 ★
Retrofit	3.7352 ± 0.439 ▲	1.5152 ± 0.0 ★	4.7297 ± 0.772 ▲	1.7941 ± 0.0 ★
XZing	39.7929 ± 0.643 ▲	1.0 ± 0.0 ★	39.1204 ± 1.701 ▲	1.0 ± 0.0 ★
IOF/ROL	1.6445 ± 0.158 ▲	1.2626 ± 0.064 ★	1.5639 ± 0.108 ▲	1.0992 ± 0.05 ★
Paint Control	1 ± 0.0	1 ± 0.0	1.001 ± 0.002 ▽	1.0 ± 0.0 ★
GSDTSR	3.2553 ± 0.387 ▲	2.1316 ± 0.1 ★	3.8958 ± 0.18 ▲	1.1482 ± 0.071 ★
Time Budget: 50%				
Druid	1225.6197 ± 600.746 ▲	121.8396 ± 43.763 ★	1420.3143 ± 418.926 ▲	51.2697 ± 11.926 ★
Fastjson	1527.9434 ± 93.004 ▲	315.8923 ± 79.836 ★	1173.9871 ± 321.789 ▲	335.9929 ± 125.629 ★
DeepLearning4j	5.0267 ± 1.827 ▲	2.5496 ± 0.011 ★	7.3264 ± 0.530 ▲	2.464 ± 0.005 ★
DSpace	19.0354 ± 0.887 ▲	5.5312 ± 1.939 ★	27.6301 ± 0.921 ▲	4.1089 ± 1.769 ★
Guava	289.1586 ± 99.054 ▲	78.6409 ± 45.928 ★	235.9919 ± 27.865 ▲	24.0869 ± 21.479 ★
OkHttp	6.203 ± 2.066 ▲	4.188 ± 0.014 ★	19.6306 ± 0.79 ▲	2.3643 ± 0.002 ★
Retrofit	5.4302 ± 0.43 ▲	2.4059 ± 0.0 ★	5.625 ± 0.415 ▲	1.4299 ± 0.0 ★
XZing	51.2576 ± 0.579 ▲	5.6 ± 0.0 ★	49.4232 ± 3.15 ▲	2.0 ± 0.0 ★
IOF/ROL	1.8009 ± 0.292 ▲	1.2588 ± 0.035 ★	1.9213 ± 0.218 ▲	1.151 ± 0.024 ★
Paint Control	1.0234 ± 0.004 ▲	1.0018 ± 0.001 ★	1.0257 ± 0.001 ▲	1.0014 ± 0.001 ★
GSDTSR	1.9072 ± 0.06 ★	2.1461 ± 0.101 ▲	3.5648 ± 0.141 ▲	1.1505 ± 0.072 ★
Time Budget: 80%				
Druid	1427.8103 ± 493.429 ▲	146.9112 ± 46.436 ★	1053.3369 ± 658.042 ▲	50.3805 ± 11.25 ★
Fastjson	1535.2998 ± 101.625 ▲	298.8445 ± 105.27 ★	993.382 ± 393.441 ▲	572.0954 ± 221.573 ★
DeepLearning4j	5.5748 ± 2.558 ▲	2.8146 ± 0.014 ★	7.8533 ± 0.581 ▲	2.5017 ± 0.011 ★
DSpace	23.126 ± 0.321 ▲	6.9419 ± 0.019 ★	33.949 ± 0.919 ▲	6.075 ± 0.34 ★
Guava	330.5342 ± 74.0 ▲	84.656 ± 34.075 ★	202.2301 ± 47.258 ▲	83.989 ± 78.446 ★
OkHttp	4.2988 ± 2.242 ▲	4.0748 ± 0.021 ★	21.5784 ± 1.148 ▲	2.282 ± 0.003 ★
Retrofit	5.9252 ± 0.884 ▲	2.4636 ± 0.0 ★	5.8832 ± 0.506 ▲	1.4386 ± 0.0 ★
XZing	51.0061 ± 1.233 ▲	4.2727 ± 0.0 ★	48.648 ± 2.926 ▲	1.1366 ± 0.0 ★
IOF/ROL	2.021 ± 0.447 ▲	1.317 ± 0.026 ★	2.2548 ± 0.362 ▲	1.2366 ± 0.017 ★
Paint Control	1.015 ± 0.001 ▲	1.0003 ± 0.001 ★	1.0344 ± 0.009 ▲	1.0003 ± 0.001 ★
GSDTSR	1.9413 ± 0.322 ★	2.1461 ± 1.011 ▲	3.4858 ± 0.012 ▲	1.1505 ± 0.072 ★

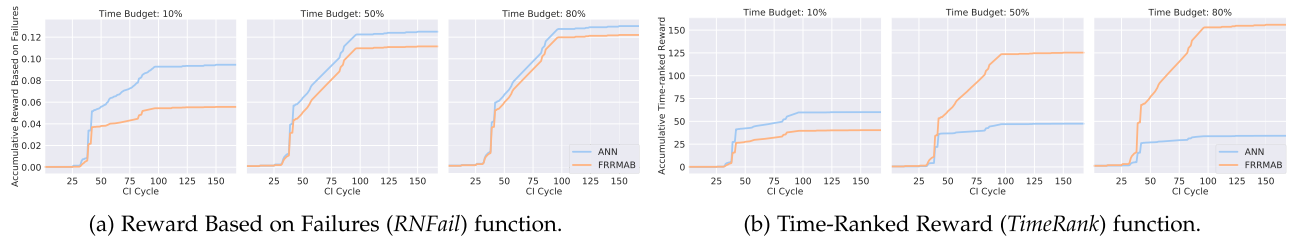


Fig. 3. Accumulative Reward values over the CI Cycles from Druid system.

the best, with statistical difference considering the time budgets of 50 and 80 percent.

We observe that ANN has a bad performance for Druid and DeepLearning4j, and improves performance in comparison with FRRMAB when the budget decreases. On the other hand, FRRMAB presents more stability in NAPFD average values than ANN. FRRMAB has the best performance with low variation and better values for Druid and DeepLearning4j. Regarding APFDc values, ANN using *RNFail* function stands out in 64 percent of the cases (21 out of 33) against 45 percent (15 cases) for FRRMAB. ANN is the best with statistical difference in ≈ 55 percent (18 out of 33) of the cases against 30 percent (10) for FRRMAB. Considering *TimeRank* the opposite occurs. FRRMAB stands out in 55 percent of the cases (18 out of 33), against 48 percent (16) for ANN. FRRMAB is the best with statistical difference in ≈ 52 percent (17 out of 33) of the cases against 42 percent (14) for ANN. Across the SUTs, overall, FRRMAB and ANN are statistically equivalent in most of the time budgets and reward functions, except in *TimeRank* function with a time budget of 80 percent in which FRRMAB is better than ANN. For this indicator, the results show the approaches are competitive, that is, there is not a great difference between them. A possible reason for this is that RETECS considers the individual test case duration, and COLEMAN, differently, considers only historical failure data.

FRRMAB with *TimeRank* produces better values concerning NAPFD, APFDc, and time budgets of 50 and 80 percent. But overall, the results of both functions are similar.

According to Table 4, ANN is a bit faster than FRRMAB considering the function *RNFail*, and slower considering *TimeRank*. FRRMAB is stable for both functions, whilst ANN spends more time using *TimeRank* function. Analyzing the prioritization time spent along with CI Cycles, we observe that both approaches spend more time when test cases are added or removed from one cycle to another. This occurs because both approaches need to update the information about the test cases, mainly when a high number of test cases increases or decreases abruptly. But a greater impact is observed for RETECS. In particular, ANN has a great variation in the spent time for IOF/ROL. For this system, RETECS can take in the worst case more than one second.

Regarding NTR, for the three budgets, FRRMAB is the best in ≈ 64 percent (21 out of 33) of the cases against 36 percent (12) for ANN for *RNFail* function. Considering *TimeRank*, FRRMAB is the best in ≈ 73 percent (24) of the cases against ≈ 27 percent (9) for ANN. Regarding RFTC (Table 6) and *RNFail*, FRRMAB stands out in ≈ 94 percent of the cases (31 out of 33) against ≈ 12 percent (4) for ANN. But in 2 of

these 4 cases, ANN and FRRMAB are statistically equivalent. With *TimeRank*, FRRMAB is the best in all cases.

With the systems used, we do not identify a scalability pattern concerning the time spent to prioritize the test cases. The use of systems with a great number of test cases in each CI Cycles can help in the identification of our approach scalability in future studies.

Due to the bad performance of ANN in Druid, we analyze this system. Druid has the lowest number of CI Cycles (168) among the systems evaluated, as well as it has many tests (2391). Fig. 3 shows the accumulative reward values from ANN (blue line) and FRRMAB (orange line) along the CI Cycles considering *RNFail* and *TimeRank* functions. We can observe that although RETECS better assigns the rewards in the *RNFail* function, the prioritization order is not adequate, which we can see with the *TimeRank* function. In this way, FRRMAB better mitigates than ANN the problem of beginning without learning mainly using the *TimeRank* function, and adapts quickly to deal with a peak of faults in the first cycles.⁵

RQ3: COLEMAN using FRRMAB outperforms RETECS using ANN regarding NAPFD, RFTC and NTR indicators, independently of the reward functions and budgets investigated. Regarding APFDc, both approaches present similar results. Besides, with respect to the execution time, FRRMAB is more stable, that is, adapts better to deal with peak of faults.

7 CONCLUDING REMARKS

This work introduces COLEMAN, an approach to deal with the TCP CI problem. COLEMAN is based on MAB with combinatorial and volatile characteristics to dynamically obtain an adequate prioritized test suite for each CI Cycle (commit) using historical failure data of test cases. The approach properly deals with the EvE dilemma and takes into account TCP CI characteristics such as test case volatility.

We evaluated our approach concerning three time budgets: 10 percent, 50, and 80; with five MAB policies: FRRMAB, UCB, ϵ -Greedy, Greedy, and Random; and compared it against an RL-based approach from literature, named RETECS [14]. The results show that FRRMAB policy outperforms the other policies with both reward functions. In most cases, our approach outperforms RETECS in terms of NAPFD, NTR, and earlier fault detection (RFTC). It also does not present great variations in the prioritization time, and does not require any additional information.

5. See supplementary material about characteristics of this system.

Regarding APFDc, the obtained values are competitive with RETECS.

Our approach spends, in the worst case, less than one second to prioritize a test suite, and the obtained results in many aspects investigated show high performance. Then, we can conclude that COLEMAN contributes efficiently and effectively to address the TCPIC problem.

Future work includes the application of our approach in other systems, possibly in an industrial scenario, as well as the study of other policies to include the individual test case duration in the prioritization. We intend to evaluate the scalability of our approach and provide a relationship between the number of test cases and the prioritization time. Another work to be conducted is to provide COLEMAN as an API or an *add-on*, as well as the source code.

ACKNOWLEDGMENTS

This work is supported by the Brazilian funding agencies CAPES and CNPq. Grant: 305968/2018.

REFERENCES

- [1] Y. Zhao, A. Serebrenik, Y. Zhou, V. Filkov, and B. Vasilescu, "The impact of continuous integration on other software development practices: A large-scale empirical study," in *Proc. 32nd IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2017, pp. 60–71.
- [2] A. Memon *et al.*, "Taming google-scale continuous testing," in *Proc. 39th Int. Conf. Softw. Eng.: Sof. Eng. Practice Track*, 2017, pp. 233–242.
- [3] M. Hilton, T. Tunnell, K. Huang, D. Marinov, and D. Dig, "Usage, costs, and benefits of continuous integration in open-source projects," in *Proc. 31st IEEE/ACM Int. Conf. Automated Softw. Eng.*, 2016, pp. 426–437.
- [4] A. Haghighatkah, M. Mäntylä, M. Oivo, and P. Kuvaja, "Test prioritization in continuous integration environments," *J. Syst. Softw.*, vol. 146, pp. 80–98, 2018.
- [5] S. Elbaum, G. Rothermel, and J. Penix, "Techniques for improving regression testing in continuous integration development environments," in *Proc. 22nd ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2014, pp. 235–245.
- [6] H. Do, S. Mirarab, L. Tahvildari, and G. Rothermel, "An empirical study of the effect of time constraints on the cost-benefits of regression testing," in *Proc. 16th ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2008, pp. 71–82.
- [7] S. Yoo and M. Harman, "Regression testing minimization, selection and prioritization: A survey," *Softw. Test. Verif. Reliab.*, vol. 22, no. 2, pp. 67–120, Mar. 2012.
- [8] C. Henard, M. Papadakis, M. Harman, Y. Jia, and Y. Le Traon, "Comparing white-box and black-box test prioritization," in *Proc. 38th Int. Conf. Softw. Eng.*, 2016, pp. 523–534.
- [9] D. Marijan, A. Gotlieb, and S. Sen, "Test case prioritization for continuous regression testing: An industrial case study," in *Proc. IEEE Int. Conf. Softw. Maintenance*, 2013, pp. 540–543.
- [10] D. Marijan, "Multi-perspective regression test prioritization for time-constrained environments," in *Proc. IEEE Int. Conf. Softw. Quality Rel. Secur.*, 2015, pp. 157–162.
- [11] D. Marijan, M. Liaaen, A. Gotlieb, S. Sen, and C. Ieva, "TITAN: Test suite optimization for highly configurable software," in *Proc. IEEE Int. Conf. Softw. Testing Verification Validation*, 2017, pp. 524–531.
- [12] Y. Cho, J. Kim, and E. Lee, "History-based test case prioritization for failure information," in *Proc. 23rd Asia-Pacific Softw. Eng. Conf.*, 2016, pp. 385–388.
- [13] B. Busjaeger and T. Xie, "Learning for test prioritization: An industrial case study," in *Proc. 24th ACM SIGSOFT Int. Symp. Foundations Softw. Eng.*, 2016, pp. 975–980.
- [14] H. Spieker, A. Gotlieb, D. Marijan, and M. Mossige, "Reinforcement learning for automatic test case prioritization and selection in continuous integration," in *Proc. 26th ACM SIGSOFT Int. Symp. Softw. Testing Anal.*, 2017, pp. 12–22.
- [15] J. A. Prado Lima and S. R. Vergilio, "Test case prioritization in continuous integration environments: A systematic mapping study," *Inf. Softw. Technol.*, vol. 121, 2020, Art. no. 106268.
- [16] R. S. Sutton and A. G. Barto, "Reinforcement learning: An introduction," Cambridge, MA, USA: MIT Press, 2011.
- [17] J. M. Benitez, J. L. Castro, and I. Requena, "Are artificial neural networks black boxes?" *Trans. Neural Netw.*, vol. 8, no. 5, pp. 1156–1164, Sep. 1997.
- [18] H. Robbins, "Some aspects of the sequential design of experiments," in *Herbert Robbins Selected Papers*. Berlin, Germany: Springer, 1985, pp. 169–177.
- [19] P. Auer, N. Cesa-Bianchi, and P. Fischer, "Finite-time analysis of the multiarmed bandit problem," *Mach. Learn.*, vol. 47, no. 2, pp. 235–256, May 2002.
- [20] V. Kuleshov and D. Precup, "Algorithms for multi-armed bandit problems," *J. Mach. Learn. Res.*, vol. 1, pp. 1–48, 2014.
- [21] J. A. Prado Lima and S. R. Vergilio, "A multi-armed bandit approach for test case prioritization in continuous integration environments," May 2020. [Online]. Available: osf.io/wmcdb
- [22] A. Fialho, "Adaptive operator selection for optimization," Ph.D. dissertation, Université Paris Sud-Paris XI, France, 2010.
- [23] K. Li, A. Fialho, S. Kwong, and Q. Zhang, "Adaptive operator selection with bandits for a multiobjective evolutionary algorithm based on decomposition," *IEEE Trans. Evol. Computation*, vol. 18, no. 1, pp. 114–130, Feb. 2014.
- [24] V. Anantharam, P. Varaiya, and J. Walrand, "Asymptotically efficient allocation rules for the multiarmed bandit problem with multiple plays-Part I: I.I.D. rewards," *IEEE Trans. Autom. Control*, vol. 32, no. 11, pp. 968–976, Nov. 1987.
- [25] Z. Bnaya, R. Puzis, R. Stern, and A. Felner, "Volatile multi-armed bandits for guaranteed targeted social crawling," in *Proc. Workshops 27th AAAI Conf. Artif. Intell.*, 2013, pp. 8–10.
- [26] P. M. Duvall, S. Matyas, and A. Glover, *Continuous Integration: Improving Software Quality and Reducing Risk*. London, U.K.: Pearson Education, 2007.
- [27] C. Catal and D. Mishra, "Test case prioritization: A systematic mapping study," *Softw. Qual. J.*, vol. 21, no. 3, pp. 445–478, Sep. 2013.
- [28] M. Khatibsyarabini, M. A. Isa, D. N. Jawawi, and R. Tumeng, "Test case prioritization approaches in regression testing: A systematic literature review," *Inf. Softw. Technol.*, vol. 93, pp. 74–93, 2018.
- [29] J. Liang, S. Elbaum, and G. Rothermel, "Redefining prioritization: Continuous prioritization for continuous integration," in *Proc. 40th Int. Conf. Softw. Eng.*, 2018, pp. 688–698.
- [30] Y. Zhu, E. Shihab, and P. C. Rigby, "Test re-prioritization in continuous testing environments," in *Proc. IEEE Int. Conf. Softw. Maintenance Evol.*, 2018, pp. 69–79.
- [31] B. Zoph and Q. V. Le, "Neural architecture search with reinforcement learning," in *5th Int. Conf. Learn. Representations*, 2017.
- [32] X. Qu, M. B. Cohen, and K. M. Woolf, "Combinatorial interaction regression testing: A study of test case generation and prioritization," in *Proc. IEEE Int. Conf. Softw. Maintenance*, 2007, pp. 255–264.
- [33] S. Elbaum, A. Malishevsky, and G. Rothermel, "Incorporating varying test costs and fault severities into test case prioritization," in *Proc. 23rd Int. Conf. Softw. Eng.*, 2001, pp. 329–338.
- [34] G. Rothermel, R. H. Untch, C. Chu, and M. J. Harrold, "Test case prioritization: An empirical study," in *Proc. IEEE Int. Conf. Softw. Maintenance*, 1999, pp. 179–188.
- [35] W. H. Kruskal and W. A. Wallis, "Use of ranks in one-criterion variance analysis," *J. Amer. Statist. Assoc.*, vol. 47, no. 260, pp. 583–621, 1952.
- [36] H. B. Mann and D. R. Whitney, "On a test of whether one of two random variables is stochastically larger than the other," *Ann. Math. Statist.*, vol. 18, pp. 50–60, 1947.
- [37] M. Friedman, "A comparison of alternative tests of significance for the problem of m rankings," *Ann. Math. Statist.*, vol. 11, no. 1, pp. 86–92, 1940.
- [38] A. Vargha and H. D. Delaney, "A critique and improvement of the CL common language effect size statistics of McGraw and Wong," *J. Educ. Behavioral Statist.*, vol. 25, no. 2, pp. 101–132, Jan. 2000.
- [39] Travis CI, "Travis CI," Accessed: Jan. 22, 2018. [Online]. Available: <https://travis-ci.org>
- [40] M. Beller, G. Gousios, and A. Zaidman, "TravisTorrent: Synthesizing Travis CI and GitHub for full-stack research on continuous integration," in *Proc. 14th Working Conf. Mining Softw. Repositories*, 2017, pp. 447–450.



Jackson Antonio do Prado Lima received the master's degree from the Federal University of Paraná (UFPR), Brazil, in 2017. Currently, he is working toward the PhD degree in computer science at UFPR. His research interests include software testing, search-based software engineering, evolutionary computation, machine learning, deep learning, and blockchains.



Silvia Regina Vergilio is currently a professor of software engineering with the Computer Science Department, Federal University of Paraná (UFPR), Brasil, where she leads the research group on software engineering. Her research interests include software testing, software reliability, software product lines (SPLs) and search-based software engineering (SBSE). She serves on the program committee of diverse conferences related to SBSE and software testing, acts as peer reviewer for diverse international journals and is assistant editor of the *Journal of Software Engineering: Research and Development*.

▷ For more information on this or any other computing topic, please visit our Digital Library at www.computer.org/csdl.